

Table of Contents

Project Documentation.....	1
Project Overview.....	3
Summary of System and Purpose.....	3
Key Features.....	3
Technologies Used.....	4
Architecture Summary.....	4
Description of 3-Tier Architecture.....	5
Cross-Cutting Concerns.....	6
MVC, DI, ORM usage.....	6
Design Artifacts.....	8
Class Diagram.....	8
Sequence Diagrams.....	9
Deployment Diagram.....	11
Package Diagram.....	12
UI Screenshots.....	12
Entity and Relationship Mapping.....	19
List of Entities.....	19
Relationships and Annotations.....	19
Cascade/Fetch/Validation Notes.....	20
Pattern Usage.....	22
Business Rules.....	23
Security and Logging.....	24
Authentication and Roles.....	24
Logging Configuration and Samples.....	25
Challenges and Learnings.....	29
Technical challenges.....	29
Personal reflections.....	30
Suggestions for improvement.....	31

Project Overview

Summary of System and Purpose

The **Come On Inn** hotel management system is designed to facilitate the automation and digitalization of bookings and administrative management of the hotel, modernizing its current mode of operation and setting the foundation for future scalability. It aims to significantly reduce the time and organizational efforts required to manage the extensive data kept by the hotel system and implement additional security protocols, quick response time, efficient data storage and fast retrieval, accurate calculation operations, and reduce the capacity for user error.

The outcome of this future mode of operation intends to increase user convenience and satisfaction, allowing automated and efficient reservation processes with built-in validation and service features, ensuring that there is less friction for the booking and checkout processes. It also intends to improve the time management and reduce employee workload, automating and improving the efficiency of the storage and retrieval of customer and hotel data, shifting the responsibilities of employees to more client-facing tasks and improving employee moral and retention by eliminating menial labour and risk of human error. Finally, the centralization of data into digital repositories ensures that all information is accounted for, simplifying the processing of financial information for customer, revenue, and audit purposes, providing a clear picture of the operating costs and revenue generation of the system and allowing stakeholders to identify areas of improvement. This also results in a significant increase to the consistency of reporting and gathering of data, eliminating errors in human calculation, data duplication, and misplaced files.

With the implementation of this new system, the **Come On Inn** hotel chain seeks to increase its competitive edge in the hotel market and expand its operations to other locations which can smoothly be integrated into this modular and highly scalable system.

Key Features

- Automated reservation booking via online kiosk with configurable pricing system and validation methods for data integrity and room availability
- Authentication service with administrative features requiring security permissions and safe password storage via hashing
- Support for role based permissions
- Configurable checkout system with support for discounts, points redemption, and partial payment support
- Robust logging and audit services, allows admins to track state of app and user interactions
- Export of data to PDF, CSV, and TXT file formats
- Feedback submission and processing for post-checkout assessment
- Object-Relational Mapping (ORM) for data storage and retrieval between the back-end database and application logic
- Admin interface panels allowing the aggregation and sorting of data
- Room occupancy management and report generation

- Loyalty Point system intended to promote guest retention by allowing the accrual of loyalty points during reservation bookings and the redemption of points during payments
- Customizable policies for earning rates and redemption of loyalty points and discount rates for guests based on admin role
- Waitlist system and notification for pending availability of rooms
- Ability to efficiently track payments and loyalty points for guests
- Standardization and centralization of data storage by using a central database with individual entities, repositories, and services, with explicitly defined relations between them
- Consistency in logic flow, data storage, and processing with clear and maintainable processes

Technologies Used

This system is built entirely with Java, using the IntelliJ IDE and SceneBuilder for UI design. Dependency Injection is handled using Guice, password security and hashing is handled by BCrypt, Hibernate framework is used for relational mapping, Simple Logging Facade for Java (SLF4J) used for logging service, H2 used as the database engine for the embeddable SQL database, and Maven used for build automation.



Maven



Architecture Summary

Description of 3-Tier Architecture



Client Tier (Views)



This tier is the User Interface (UI) consisting of all the FXML views in the application. This is the presentation layer displayed to the users of the application and is where the interactions with the application take place. Primarily this consists of all the containers and controls. The containers specify the layout arrangement of various controls to ensure appropriate positioning, flexibility for different viewport sizes, and consistency of appearance. These tend to be comprised of GridPane, StackPane, and BorderPane elements. GridPanes are commonly used throughout the app in form components or data presentation, such as the 5-step reservation booking form and the checkout information. BorderPanes provide the standard layout of the app components, usually depicting a header and footer banner with a central panel containing the page controls and an optional side panel containing button navigation. These are most frequently the anchor element or root of the application's views, as other containers are often nested inside of a BorderPane. StackPanes are used as the central parent container in a BorderPane root, allowing the user to cycle through various different panes inside of the StackPane by toggling their visibilities via button navigation. This is done in the self-serve kiosk for reservation booking, with each step defined inside a container within the StackPane and navigable through the "Next" and "Previous" buttons, allowing a more efficient and modular form logic and sequential validation. A StackPane is also used in the admin view, allowing administrative users to cycle through different types of data by using the side panel buttons. In both cases the aim is to not overwhelm the user by cluttering a single screen with too much information, making it more easily digestible by grouping it and displaying only a single aspect at a time.

Application Tier (Controllers)

This layer acts as an intermediary between the front-end client tier and the back-end data tier and handles all of the core logic and processing of the application. Through the client-tier user interface, a request is made to create, retrieve, update or delete certain data (CRUD) from the back-end, and this tier processes these requests and handles how the returned data should be manipulated and configured to display back to the user. This is handled by the controller classes, utilities, policies, and services of the application. This tier defines the business rules and acts as the core of the application where its main functionality resides.

Data Tier (Models)

This is the back-end bottommost tier which manages the data of the application. It defines the models used to store data and responds to requests from the client views via the application tier to store, retrieve, and manage this data. This is comprised of all the entities in this application (see section 4 of the document) and the H2 SQL database. The hotel management system uses a relational database management system (RDMS) with clearly defined relationships and organization of the data. This entails specifying the tables, columns, constraints and primary keys, and storage of objects into single rows of data adhering to the configuration of the database.

Cross-Cutting Concerns

These are the components of the hotel management system which do not abide by the object oriented principles of modularity and encapsulation and impact several different modules. Their configurations are necessary but raise concerns about potential problems in the design such as inter-dependency tangling, scattering of code, and code duplication. These include logging, policies and security, validation, and database operations. In this context, logging is comprised of both recording application events for debugging purposes (using the Logger dependency to export to log files) and using the Audit Log model to store and monitor administrative events for auditing purposes. Both of these are integral to the operation of the system and are fundamental to identifying issues in a timely and efficient manner and providing historical records and contexts to actions being performed by both the application logic processes and the users. The former is also closely tied to error handling and exceptions to ensure an archive and stack trace of the error is logged and exported to file, and the latter is connected to the monitoring system of alert objects to provide visual response to the user after an action is performed. Data validation is also a crucial component of the application and addresses the core issues of the previous file-based mode of operations by ensuring the integrity of the data being stored and preventing situations that would often previously arise due to human error such as overbooking, mismanagement of reservations or finances, and storage of incorrect data. This clearly defines the business rules of the application, the expectations for the user in entering data during processes such as the 5-step reservation booking kiosk, defining the expectation of types and values for the data, and the conformity of the data to the back-end Schema structure. In future implementations, this can also be scaled to account for concurrency problems by ensuring that the data is synchronized and locks are in place to control access to shared resources so that the data operations can be atomized.

MVC, DI, ORM usage

The design philosophy of the hotel management system adheres to Model-View-Controller (MVC) principles as defined in the 3-Tier architecture summary. All user interface (UI) elements are presented in FXML views, with controller classes handling view bindings and logic for processing requests and manipulating data. Models are defined to represent the structure of the data and handle getter and setter methods, with repositories used to store collections. Each aspect of the MVC architectural

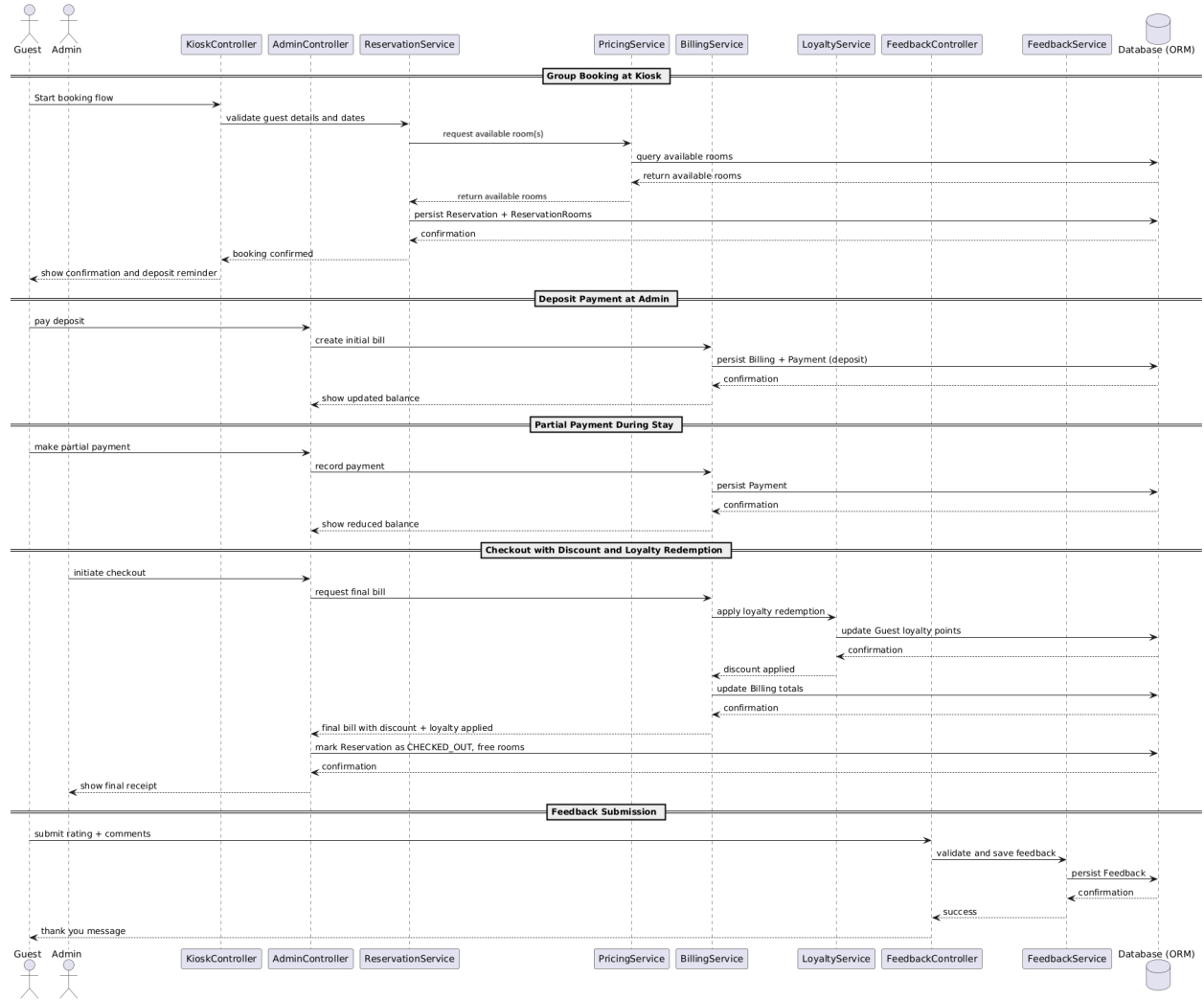
design pattern is grouped and named in its specific package (models/views/controllers), separating the logic accordingly and ensuring that the classes and application components are organized effectively.

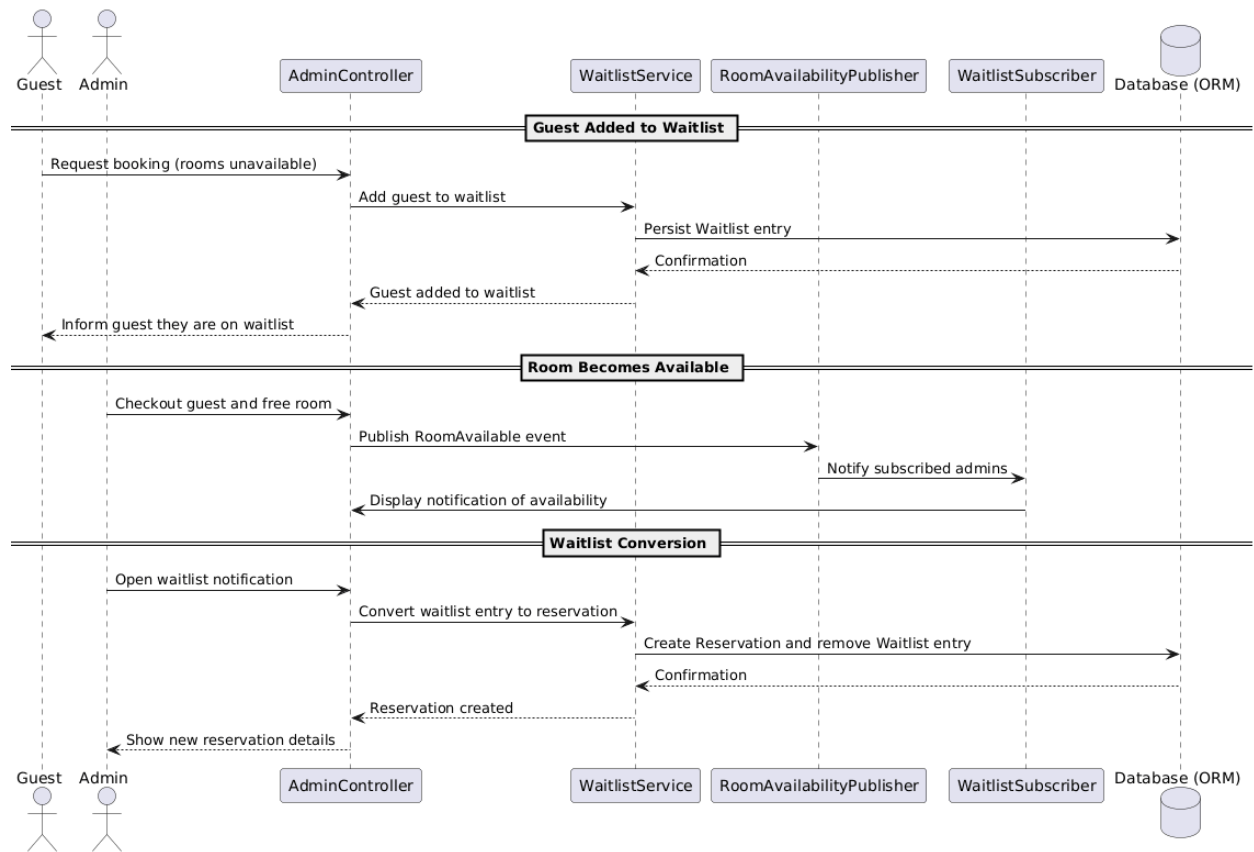
Dependency Injection (DI) is used throughout the app to ensure that classes do not instantiate their own repositories, services, or entity managers, but receive them from external sources using injection. This decouples their implementation from these dependencies and ensures consistency and single-use, so that (for example) each controller of the application does not use its own model repository but shares a singleton repository where all the model data for a specific model is stored. To accomplish this, the hotel management system relies on the open-source dependency framework Guice, which defines its bindings in the GuiceModule file. The repositories or services are then injected into the appropriate classes which require them, reducing coupling, ensuring reusability, and centralization of data.

Object-Relational Mapping (ORM) is used throughout the app to map the model classes to relational database tables, using Jakarta Persistence (JPA). This is a core aspect of the application as it allows for persistence in the data, conversion of objects into database rows, and is crucial to each of the various processes of the application without which they would not be able to function. A persistence unit (hotelPU) is defined in the META-INF/persistence.xml file, which also specifies all of the model classes used by the application along with other various important properties (such as defining the auto process of hibernate to update the existing persistent database instead of creating a new one on each initialization of the application). Model classes are defined with the “@Entity” tag, specifying the table names, any unique constraints, autogenerating the table row primary keys (using “@Id” and “@GeneratedValue(strategy = GenerationType.IDENTITY)” tags) and identifying each of the model class attributes as columns using the “@Column” tag. Any relations between other models are also explicitly stated using “@JoinToColumn” and specifying their UML relations (“@ManyToOne”, “@OneToOne”, etc.). Validation is also assisted here by identifying which columns are allowed to contain null values, ensuring the integrity of the data being stored. An EntityManagerProvider class is used to instantiate a single EntityManagerFactory for the “hotelPU” persistence unit, and an EntityManager is created for each transaction made to the database tables.

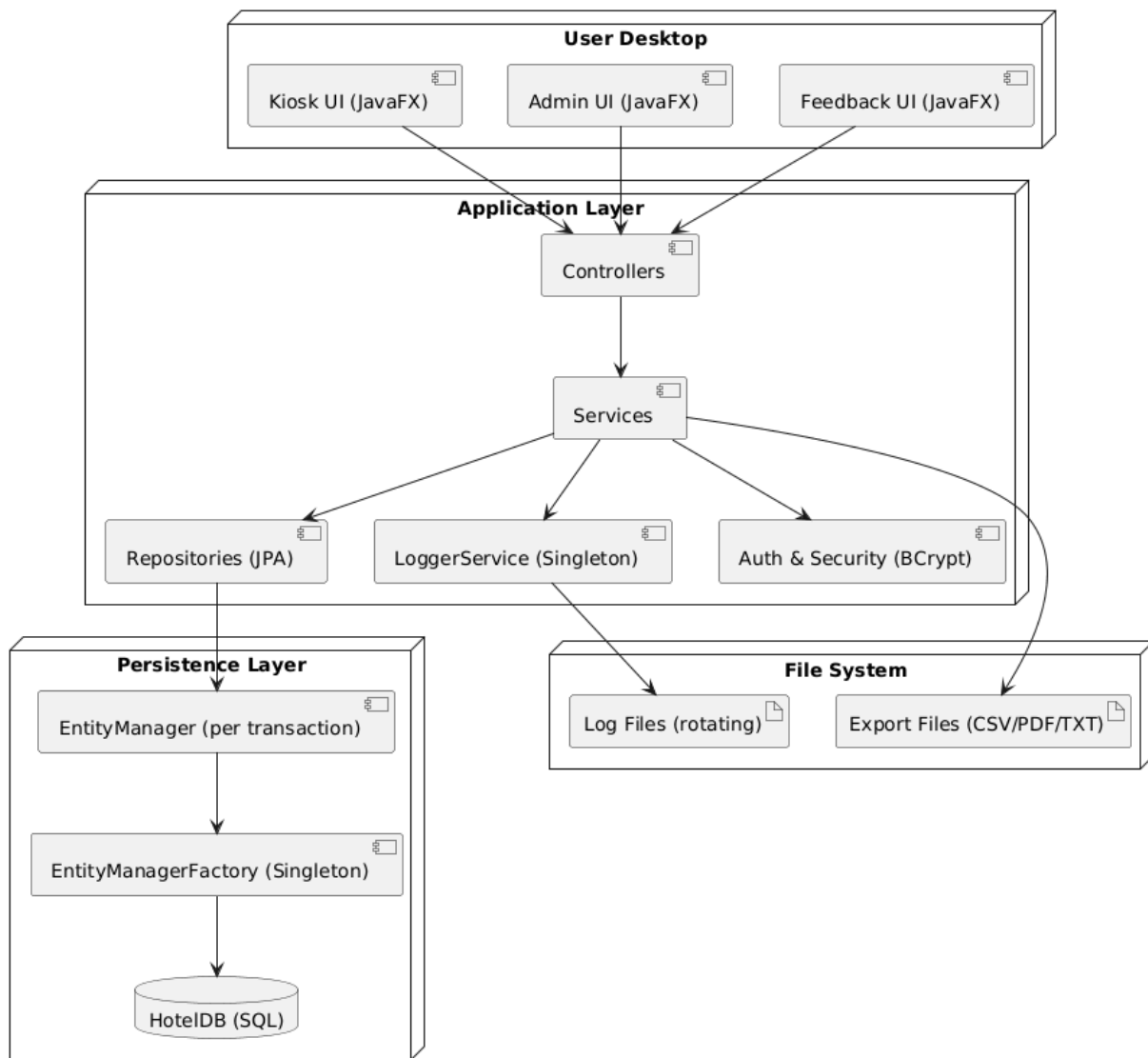
Sequence Diagrams



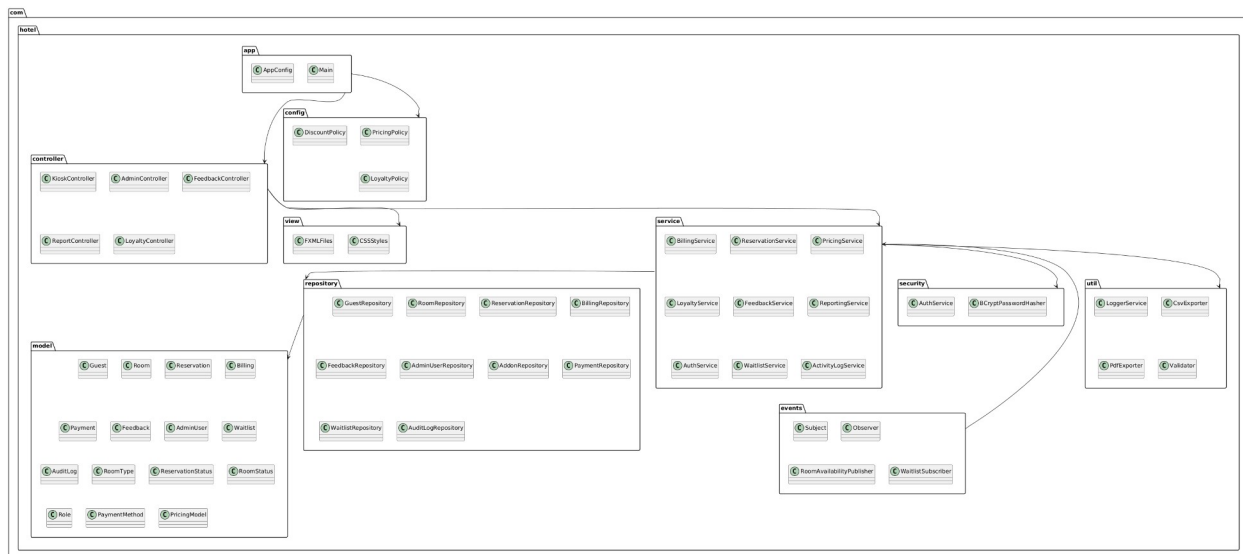




Deployment Diagram



Package Diagram



UI Screenshots



Figure 1 – Welcome Screen

The screenshot shows a window titled "Regulations" with a light blue header bar. The main content area has an orange background. At the top, the text "HOTEL REGULATIONS" is displayed in large, bold, black capital letters. Below this, the section "Room Prices" is underlined in bold. It lists four room types with their respective prices: "SINGLE (1 BED) : \$50 / NIGHT", "DOUBLE (2 BED) : \$50 / NIGHT", "DELUXE (1 BED) : \$100 / NIGHT", and "PENTHOUSE SUITE (1 BED) : \$400 / NIGHT". The next section, "Room Capacity", is also underlined in bold and lists "SINGLE, DELUXE, & PENTHOUSE - 2 PEOPLE" and "DOUBLE - 4 PEOPLE". The "Additional Services" section is underlined in bold and lists "Wi-Fi : \$10 / NIGHT", "Breakfast : \$5 / NIGHT PER PERSON", "Parking : \$20 / NIGHT", and "Spa Access : \$40 PER PERSON". On either side of the "Additional Services" section, there is a small icon of a hotel building with four stars above it.

Regulations

HOTEL REGULATIONS

Room Prices

SINGLE (1 BED) : \$50 / NIGHT DOUBLE (2 BED) : \$50 / NIGHT DELUXE (1 BED) : \$100 / NIGHT

PENTHOUSE SUITE (1 BED) : \$400 / NIGHT

Room Capacity

SINGLE, DELUXE, & PENTHOUSE - 2 PEOPLE

DOUBLE - 4 PEOPLE

Additional Services

Wi-Fi : \$10 / NIGHT

Breakfast : \$5 / NIGHT PER PERSON

Parking : \$20 / NIGHT

Spa Access : \$40 PER PERSON

Figure 2 - Regulations Screen

The screenshot shows a window titled "Administrator Login" with a light blue header bar. The main content area has a white background. At the top, the text "Administrator Login" is displayed in bold, black capital letters. Below this, there is a light orange rectangular box containing two input fields. The first field is labeled "Username :" and contains the text "Bob". The second field is labeled "Password :" and contains the placeholder text "Enter password". Below the input fields, there are two buttons: "Login" and "Cancel".

Administrator Login

Username :

Password :

Login Cancel

Figure 3 - Login Screen

Customer Kiosk

Guest Interface (c) 2025 All Rights Reserved



Book a Reservation

Checkout

Regulations

Log Out

Book a Reservation Step 1 of 5

Specify Number of Adults :

Specify Number of Children (Under 12) :

Specify Date of Check-In : 

Specify Date of Check-Out : 

Next

Figure 4 - Customer Self-Service Kiosk depicting the 5-step reservation booking process

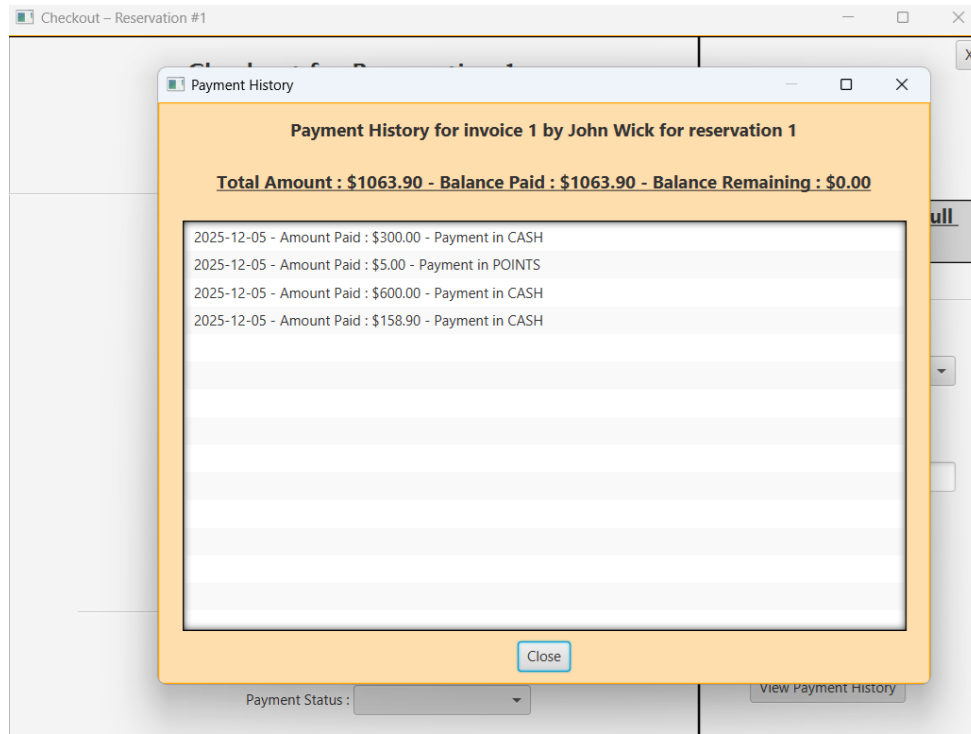


Figure 8 - Payment history modal window

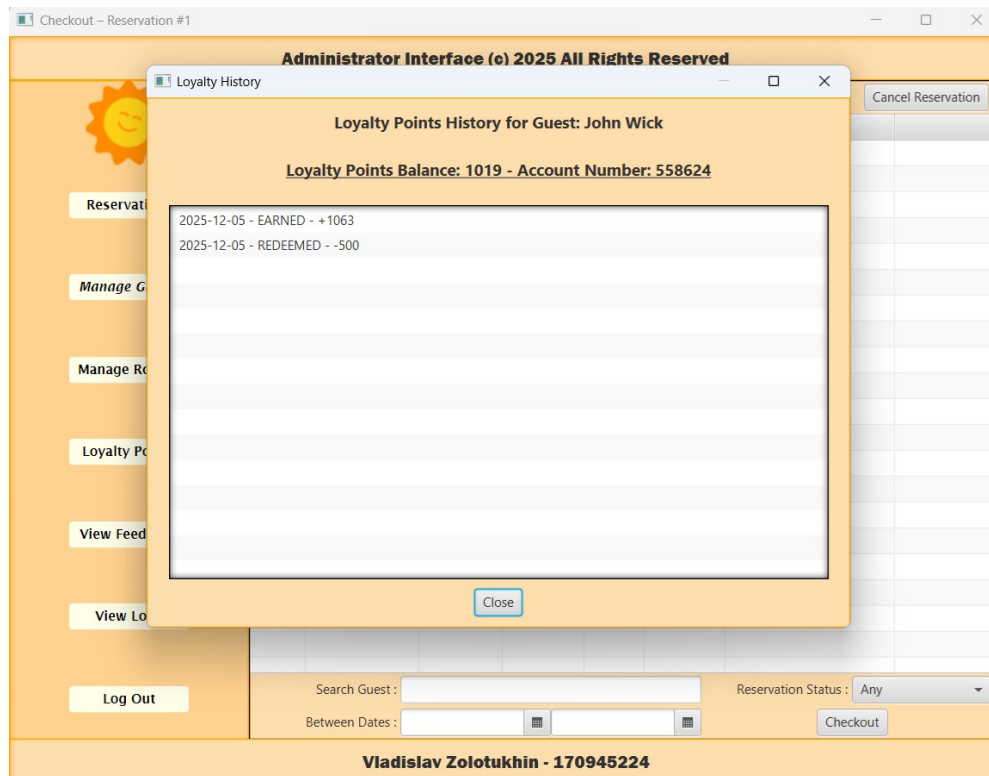


Figure 9 - Loyalty points history modal window

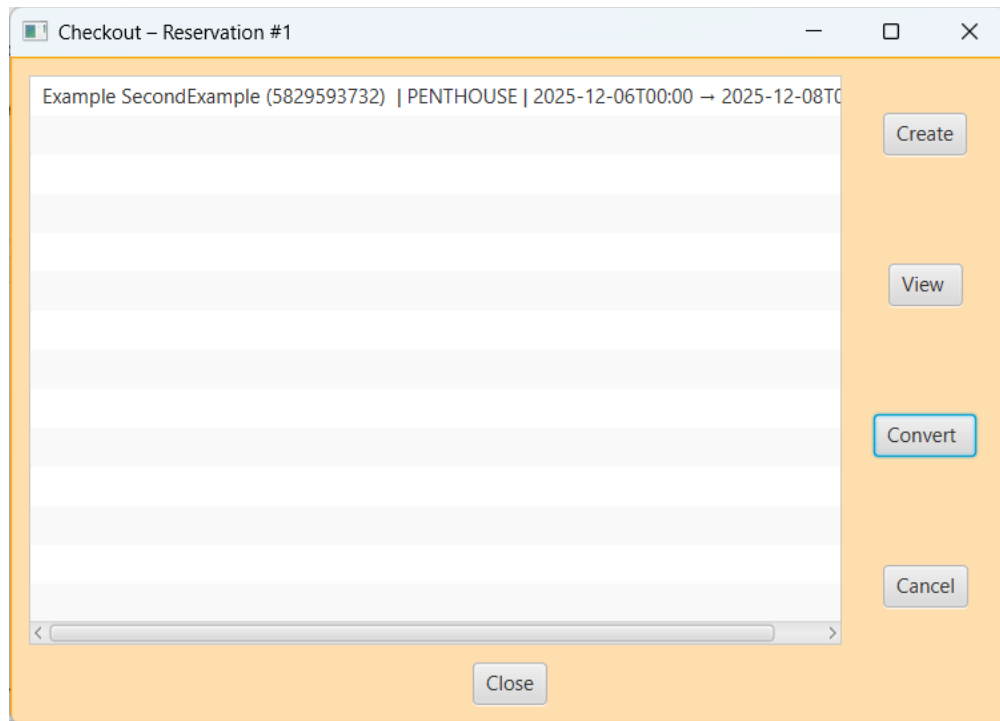


Figure 10 - Waitlist view

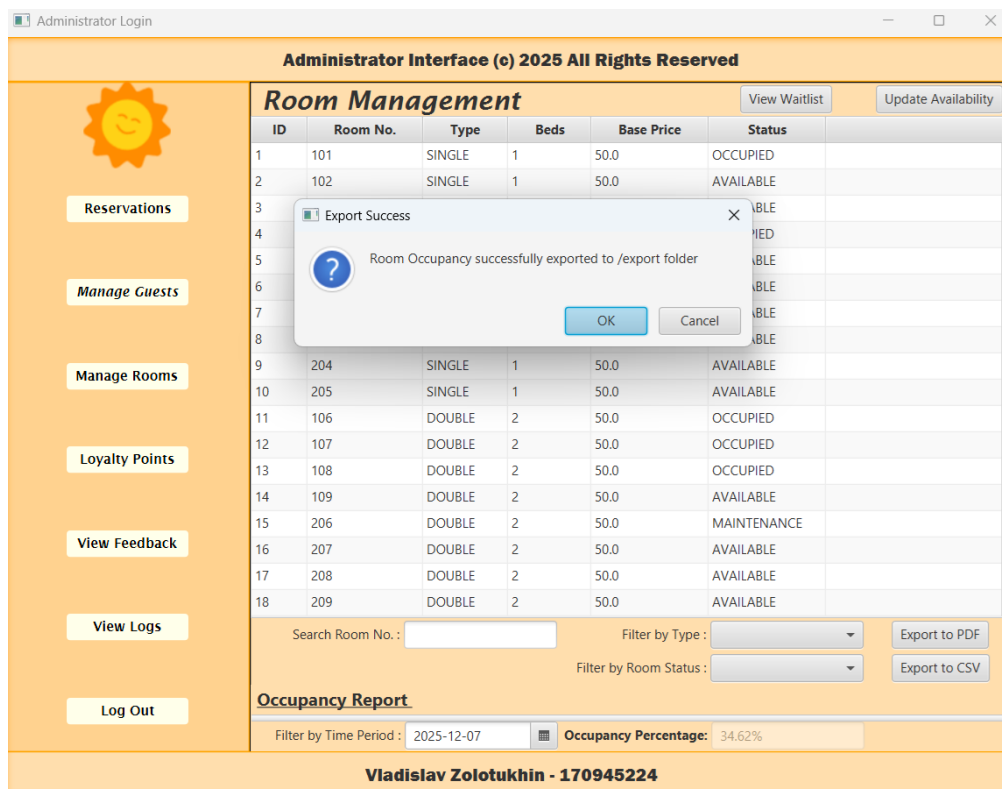


Figure 11 - Export confirmation window

Entity and Relationship Mapping

List of Entities

- **AdminUser** : id (long), username (string), passwordHash (string), role (Role), active (boolean)
- **AuditLog** : id (long), timestamp (localDateTime), actor (string), action (string), entityType (string), entityId (int), message (string)
- **Billing** : id (long), reservation (Reservation), subtotal (double), taxRate (double), taxAmount (double), discountValue (double), loyaltyRedeemedPoints (int), totalAmount (double), paidAmount (double), balanceAmount (double), paymentStatus (string)
- **Feedback** : id (long), guest (Guest), reservation (Reservation), rating (int), comments (string), sentimentTag (string), createdAt (localDateTime)
- **Guest** : id (long), name (string), phone (string), email (string), address (string), loyaltyPoints (int), loyaltyNumber (int), isActive (Boolean)
- **Hotel** : id (long), name (string), city (string)
- **Payment** : id (long), billing (Billing), method (PaymentMethod), amount (double), createdAt (localDateTime)
- **Reservation** : id (long), checkIn (localDateTime), checkout (localDateTime), numAdults (int), numChildren (int), reservationStatus (ReservationStatus), guest (Guest)
- **ReservationAddon** : id (long), reservation (Reservation), addon (ServiceAddon), quantity (int)
- **ReservationRoom** : id (long), reservation (Reservation), room (Room)
- **Room** : id (long), roomNumber (int), roomType (RoomType), beds (int), basePrice (double), roomStatus (RoomStatus)
- **ServiceAddon** : id (long), name (string), price (double), pricingModel (PricingModel)
- **Waitlist** : id (long), guest (Guest), requestedType (RoomType), dateRangeStart (localDateRange), dateRangeEnd (localDateRange), status (string)

Relationships and Annotations

- Billing to Reservation: @OneToOne (Join on reservation_id)
- Feedback to Guest: @ManyToOne (Join on guest_id)
- Feedback to Reservation: @OneToOne (Join on reservation_id)
- Payment to Billing: @ManyToOne (Join on billing_id)
- Reservation to Guest: @ManyToOne (Join on guest_id)
- ReservationAddon to Reservation: @ManyToOne (Join on reservation_id)
- ReservationAddon to ServiceAddon: @ManyToOne (Join on addon_id)
- ReservationRoom to Reservation: @ManyToOne (Join on reservation_id)
- ReservationRoom to Room: @ManyToOne (Join on room_id)
- Waitlist to Guest: @ManyToOne (Join on guest_id)

Cascade/Fetch/Validation Notes

Validation:

- **AdminUser:**
 - o **Back-end validation:** Username is unique, not null, and length 200. PasswordHash, isActive and Role not null. Role length 10. Id is PK (auto-generated).
 - o **Front-end validation:** Username and password fields must not be null in login.
- **Audit-Log:**
 - o **Back-end validation:** Timestamp, actor, action, entityType, entityId all not null. Timestamp and actor length 200, action length 250, message length 300. Id is PK.
 - o **Front-end validation:** All fields automatically populated during audit log generation. In cases where the entityType being acted on is a Policy (such as LoyaltyPolicy or DiscountPolicy if the administrative action is applying a discount rate or changing the earning rate/redemption cap for loyalty points) and not a model, the entityId value is autogenerated to 0 to provide a valid value.
- **Billing:**
 - o **Back-end validation:** Reservation, subTotal, taxRate, taxAmount, totalAmount, balanceAmount and paymentStatus all not null. PaymentStatus length 200. Reservation must be unique (exhibits One to One relation with Billing). Id is PK.
 - o **Front-end validation:** Payment calculation automated based on policy and reservation rooms and services. Business rules state discount rate is capped at 15% for admins and 30% for managers, enforced by choice box selection capped at 5% intervals.
- **Feedback:**
 - o **Back-end validation:** Reservation must be unique and not null (exhibits One to One relation with Feedback). Comments length 300, sentimentTag length 50. Id is PK.
 - o **Front-end validation:** User can't confirm feedback form submission unless rating, sentimentTag, and comment fields are present.
- **Guest:**
 - o **Back-end validation:** Name, phone, address not null. Name length 30, email length 300. Phone length 100. ID is PK.
 - o **Front-end validation:** Name (first + last), phone, and address (street + city + country + postalcode) must be filled out. LoyaltyPoints, loyaltyNumber, and isActive are auto-generated for new guests. LoyaltyPoints can't be negative after redemption calculations during checkout. Phone length must be 10 characters. Email must follow regex pattern of “*@*”
- **Hotel:**
 - o **Back-end validation:** All fields not null. Name and city length 200. ID is PK.
 - o **Front-end validation:** N/A (auto-generated during initialization if empty)
- **Payment:**
 - o **Back-end validation:** All fields not null. Method length 20. ID is PK.
 - o **Front-end validation:** Payment method must be selected and amount specified when making a payment. CreatedAt auto-generated. Billing auto-associated with Payment.
- **Reservation:**
 - o **Back-end validation:** CheckIn, checkOut, reservationStatus and Guest not null. ReservationStatus length 20. ID is PK.

- **Front-end validation:** Checkin/checkout dates, numChildren and numAdults must be specified. ReservationStatus automatically set during reservation booking. Guest instantiated (if not already exist based on phone lookup) during booking (lookUp otherwise) and automatically associated with the reservation during booking.
- **ReservationAddon:**
 - **Back-end validation:** All fields not null. Reservation and AddonService do not have to be unique (exhibit Many to One relation). ID is PK.
 - **Front-end validation:** Cannot be a negative value.
- **ReservationRoom:**
 - **Back-end validation:** All fields not null. Reservation and Room do not have to be unique (exhibit Many to One relation). ID is PK.
 - **Front-end validation:** Ensure availability of Room before instantiation. Ensure Room capacity meets valid criteria before instantiation. Automatically generated based on user Room selection during reservation booking.
- **Room:**
 - **Back-end validation:** All fields not null. RoomNumber and basePrice length 20. RoomType length 10. RoomNumber must be unique. ID is PK
 - **Front-end validation:** Rooms automatically populated during app initialization if rooms table not empty. Only RoomStatus field can be interacted with by admin (if RoomStatus is available, can be changed to maintenance, and vice-versa)
- **ServiceAddon:**
 - **Back-end validation:** All fields not null. Name length 100. PricingModel length 30. Name must be unique. ID is PK.
 - **Front-end validation:** N/A. Automatically populated during app initialization if table not empty.
- **Waitlist:**
 - **Back-end validation:** All fields not null. RequestedType length 20. Status length 200. Guest does not have to be unique (exhibits Many to One relation). ID is PK.
 - **Front-end validation:** Same as Guest and Reservation validation.

Cascade:

If a Reservation is cancelled or deleted, all ReservationAddon and ReservationRooms are deleted. Practically speaking, this means the availability of the Rooms is reset and they immediately become available for other bookings during the dates in which they were formerly occupied. Otherwise, there are no specific cascades and persist, merge, and remove operations performed on single entities only.

Fetch: All data is loaded immediately when required.

Pattern Usage

Dependency Injection:

```
bind(GuiceFXMLLoader.class).in(Singleton.class);
```

Repositories:

```

bind(AddonRepository.class).to(AddonRepositoryImpl.class).in(Singleton.class);
bind(AdminUserRepository.class).to(AdminUserRepositoryImpl.class).in(Singleton.class);
bind(AuditLogRepository.class).to(AuditLogRepositoryImpl.class).in(Singleton.class);
bind(BillingRepository.class).to(BillingRepositoryImpl.class).in(Singleton.class);
bind(FeedbackRepository.class).to(FeedbackRepositoryImpl.class).in(Singleton.class);
bind(GuestRepository.class).to(GuestRepositoryImpl.class).in(Singleton.class);
bind(PaymentRepository.class).to(PaymentRepositoryImpl.class).in(Singleton.class);
bind(ReservationRepository.class).to(ReservationRepositoryImpl.class).in(Singleton.class);
bind(RoomRepository.class).to(RoomRepositoryImpl.class).in(Singleton.class);
bind(WaitlistRepository.class).to(WaitlistRepositoryImpl.class).in(Singleton.class);
bind(ReservationRoomRepository.class).to(ReservationRoomRepositoryImpl.class).in(Singleton.class);
bind(ReservationAddonRepository.class).to(ReservationAddonRepositoryImpl.class).in(Singleton.class);

```

Services:

```

bind(AuthService.class).to(AuthServiceImpl.class).in(Singleton.class);
bind(ReservationService.class).to(ReservationServiceImpl.class).in(Singleton.class);
bind(BillingService.class).to(BillingServiceImpl.class).in(Singleton.class);
bind(LoyaltyService.class).to(LoyaltyServiceImpl.class).in(Singleton.class);
bind(ReportingService.class).to(ReportingServiceImpl.class).in(Singleton.class);
bind(ActivityLogService.class).to(ActivityLogServiceImpl.class).in(Singleton.class);
bind(WaitlistService.class).to(WaitlistServiceImpl.class).in(Singleton.class);

```

Observer:

```

bind(WaitlistObserverImpl.class).in(Singleton.class);
bind(Observable.class).in(Singleton.class);

```

Used for waitlist throughout the app when reservations are checked out, cancelled, or waitlist is created or cancelled. Handled by AdminController to display notifications.

Factory:

```

public class EntityManagerProvider implements Provider<EntityManager> {

    private static final EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("hotelPU");
    // Make sure this matches persistence.xml

    @Override
    public EntityManager get() {
        return emf.createEntityManager();
    }
    public static EntityManagerFactory getFactory() {
        return emf;
    }
}

private static SessionFactory sessionFactory;

private HibernateUtil(){}

```

```

public static synchronized SessionFactory buildSessionFactory(){
    if(sessionFactory != null){
        return sessionFactory;
    }
}

```

Factories set in EntityManagerProvider and HibernateUtil, used in initialization. Singletons used for the entity manager, repositories, and services, binding the interfaces to the implementations. Injected into controllers when necessary.

LoyaltyPolicy used to configure global earning rates and redemption caps, can be customized by an administrative user from the admin interface view. Note that changes these values results in the auto-generation of an Audit Log to create a historical record.

DiscountPolicy used to configure the max discount per role (ADMIN or MANAGER) and to apply discounts to a Billing total by an admin who matches the role criteria.

PricingPolicy defines constants for various prices and rates, used to centralize these values as they are referenced during various processes (booking a reservation, calculating the billing...). Supports getter and setter methods for future scalability.

Business Rules

Can be viewed in application from the “Regulations” view from the welcome screen or customer kiosk. These rules state the following constants:

- **Rooms:**
 - **Single:** Price is \$50 per night, max. capacity of 2 guests per room
Available Rooms: 101, 102, 103, 104, 105, 201, 202, 203, 204, 205
 - **Double:** Price is \$50 per night, max capacity of 4 guests per room
Available Rooms: 106, 107, 108, 109, 206, 207, 208, 209
 - **Deluxe:** Price is \$100 per night, max capacity of 2 guests per room
Available Rooms: 110, 111, 112, 210, 211, 212
 - **Penthouse:** Price is \$400 per night, max capacity of 2 guests per room.
Available Rooms: 301, 302
- **Addons:**
 - **Wi-Fi:** \$10 per night
 - **Breakfast:** \$5 per night per person
 - **Parking:** \$20 per night
 - **Spa Access:** \$40 per reservation per person
- **Loyalty:**
 - **Earning Rate:** \$1 spent = 1 point earned (default), customizable to \$2 spent = 1 point earned (50%) or \$1 spent = 2 points earned (200%) by administrators
 - **Redemption:** 100 loyalty points spent = \$1 off

- **Cap:** Loyalty points redemption capped at 1000 loyalty points per reservation billing (default), can be configured up to 2000 by administrators.
- **Feedback:**
 - Feedback can be submitted by a guest after they have checked out of the reservation
 - Admin receives a notification to prompt the guest to fill out a feedback form for their stay
 - More than one feedback form cannot be submitted per reservation

Business Rules are enforced by front-end validation and back-end validation for data integrity and consistency (see validation notes). Reconfiguration of rates that apply business rules must generate an audit log associating the administrative user with the change and providing a summary of the change.

Security and Logging

Authentication and Roles

Authentication handled by the Authentication Service module (AuthService), passwords automatically hashed by BCrypt when stored in database and login compares the value to the hash. Authenticated user identification persists for the duration of their logged in session, generating activity logs for administrative actions performed by the user. Currently, administrative users can have **ADMIN** and **MANAGER** roles, which affect the types of configurations they are authorized to make to the hotel management system.

The administrative interface must not be accessible without prior authentication and is restricted to administrative users. Administrative users and regular users may access the customer kiosk, feedback submission, and regulations freely without permissions. Note that submission of feedback forms requires guests to authenticate and confirm their reservation by providing their personally identifiable information (phone number) and reservation dates.

Logging Configuration and Samples

1. Debug Logs

Configured to export to the /logs/ directory in root.

Configuration set in the LoggerService module.

Naming convention follows the "logs/system_logs.%g.log" pattern.

Log writing configuration set to automatically append existing logs.

Log size cannot exceed 1MB.

Logs rotate out once they have reached the 1MB limit. New information is written to the newly created log file (following the naming convention).

Maximum limit of 10 logs. Once this limit is reached, the system will begin overwriting past logs starting with the oldest.

Sample debug log (*system_logs.0.log*):

```
Dec. 04, 2025 3:54:29 P.M. ca.senecapolytechnic.application.apd545project.AppConfig initializeDatabase
INFO: Hotel table already populated, skipping initialization.
Dec. 04, 2025 3:54:29 P.M. ca.senecapolytechnic.application.apd545project.AppConfig initializeRooms
INFO: Rooms table already populated, skipping room initialization.
Dec. 04, 2025 3:54:29 P.M. ca.senecapolytechnic.application.apd545project.AppConfig initializeDatabase
INFO: ServiceAddons table already populated, skipping initialization.
Dec. 04, 2025 3:54:29 P.M. ca.senecapolytechnic.application.apd545project.controllers.LoginController activateAdmin
INFO: Logged in as: admin
Dec. 04, 2025 3:54:29 P.M. ca.senecapolytechnic.application.apd545project.controllers.LoginController activateAdmin
INFO: Role: ADMIN
```

2. Exported Logs

Administrators can export audit logs, feedback form submissions, room occupancy reports, and other tabular information from the panes in the administrative screen.

System supports exports to CSV, PDF, and TXT files.

Information is formatted accordingly based on the chosen extension type.

All exported files identified by type of information contained (e.g. the entity model) and appended by timestamp of export.

Sample export (Feedback-20251204-155432.csv):

guest	rating	sentiment	message	reservation	date
Agent Smith	4	Good	I had a good time.	2	2025-12-03T20:44:34.659473
Bob Dole	5	Positive	Great!!	3	2025-12-03T21:06:42.284191

Sample export (Audit Logs-20251204-153549.txt):

```
-----| Audit Log |-----
Time: 2025-12-04T01:28:16.551683
Actor: admin (ADMIN)
Action: LOGIN
Entity: AdminUser
Entity ID: 1
Message: This admin has successfully logged in
-----
```

```
-----| Audit Log |-----
Time: 2025-12-04T01:28:46.594770
Actor: admin (ADMIN)
Action: LOGOUT
Entity: AdminUser
Entity ID: 1
```

Message: This admin has logged out

-----| Audit Log |-----

Time: 2025-12-04T01:28:52.555524

Actor: admin (ADMIN)

Action: LOGIN

Entity: AdminUser

Entity ID: 1

Message: This admin has successfully logged in

-----| Audit Log |-----

Time: 2025-12-04T01:29:03.104040

Actor: admin (ADMIN)

Action: APPLIED_DISCOUNT

Entity: DiscountPolicy

Entity ID: 0

Message: Applied discount rate of %0.0 to Billing ID 1

-----| Audit Log |-----

Time: 2025-12-04T01:29:15.286036

Actor: admin (ADMIN)

Action: CONFIRMED_PAYMENT

Entity: Billing

Entity ID: 1

Message: Confirmed payment of \$500.0 (CARD) for Billing ID 1

-----| Audit Log |-----

Time: 2025-12-04T01:32:33.618742

Actor: admin (ADMIN)

Action: LOGIN

Entity: AdminUser

Entity ID: 1

Message: This admin has successfully logged in

-----| Audit Log |-----

Time: 2025-12-04T01:34:31.147212

Actor: admin (ADMIN)

Action: UPDATED_EARNING_RATE

Entity: LoyaltyPolicy

Entity ID: 0

Message: Changed loyalty point earning rate to 0.5

-----| Audit Log |-----

Time: 2025-12-04T01:34:50.718531

Actor: admin (ADMIN)

Action: UPDATED_EARNING_RATE

Entity: LoyaltyPolicy

Entity ID: 0

Message: Changed loyalty point earning rate to 1.0

-----| Audit Log |-----

Time: 2025-12-04T14:41:29.010663

Actor: admin (ADMIN)

Action: UPDATED_REDEMPTION_CAP

Entity: LoyaltyPolicy

Entity ID: 0

Message: Changed loyalty point redemption cap to 2000

-----| Audit Log |-----
Time: 2025-12-04T14:42:03.144359
Actor: admin (ADMIN)
Action: UPDATED_REDEMPTION_CAP
Entity: LoyaltyPolicy
Entity ID: 0
Message: Changed loyalty point redemption cap to 1000

-----| Audit Log |-----
Time: 2025-12-04T14:42:12.802232
Actor: admin (ADMIN)
Action: UPDATED_EARNING_RATE
Entity: LoyaltyPolicy
Entity ID: 0
Message: Changed loyalty point earning rate to 1.0

-----| Audit Log |-----
Time: 2025-12-04T15:35:41.480435
Actor: admin (ADMIN)
Action: LOGOUT
Entity: AdminUser
Entity ID: 1
Message: This admin has logged out

Sample Export (OccupancyReport-20251205-001110.pdf):

Room Occupancy Report for 2025-12-05

Generated: 2025-12-05T00:11:10.131342

ID	Room Number	Room Type	Beds	Base Price	Occupancy Status
1	101	SINGLE	1	50.0	OCCUPIED
2	102	SINGLE	1	50.0	AVAILABLE
3	103	SINGLE	1	50.0	AVAILABLE
4	104	SINGLE	1	50.0	AVAILABLE
5	105	SINGLE	1	50.0	AVAILABLE
6	201	SINGLE	1	50.0	AVAILABLE
7	202	SINGLE	1	50.0	AVAILABLE
8	203	SINGLE	1	50.0	AVAILABLE
9	204	SINGLE	1	50.0	AVAILABLE
10	205	SINGLE	1	50.0	AVAILABLE
11	106	DOUBLE	2	50.0	AVAILABLE
12	107	DOUBLE	2	50.0	AVAILABLE
13	108	DOUBLE	2	50.0	AVAILABLE
14	109	DOUBLE	2	50.0	AVAILABLE
15	206	DOUBLE	2	50.0	AVAILABLE
16	207	DOUBLE	2	50.0	AVAILABLE
17	208	DOUBLE	2	50.0	AVAILABLE
18	209	DOUBLE	2	50.0	AVAILABLE
19	110	DELUXE	1	100.0	AVAILABLE
20	111	DELUXE	1	100.0	AVAILABLE
21	112	DELUXE	1	100.0	AVAILABLE
22	210	DELUXE	1	100.0	AVAILABLE
23	211	DELUXE	1	100.0	AVAILABLE
24	212	DELUXE	1	100.0	AVAILABLE
25	301	PENTHOUSE	1	400.0	AVAILABLE
26	302	PENTHOUSE	1	400.0	AVAILABLE

Occupancy Rate: 3.8461538461538463

Challenges and Learnings

Technical challenges

There were substantial technical challenges involved in this project during every aspect of its development lifecycle. I will describe them in detail according to chronological order as best I can remember and explain the approaches I have taken to resolve them:

- 1) **Creation of the Front-End:** This project encompasses a very large scope, and it was difficult to determine what features I need to support and how they can best be represented by the controls in my views, how many views I need, how they will connect to each other, and so on. I attempted to establish a consistent design pattern to adhere to, consisting of using border panes as roots for the starting points of my views, centralizing the most important content, and using same layout organizations where applicable (for example, always a central table to show data throughout the various admin panes, having a button panel on the left side to control navigation, always using grid panes for forms and so on...). As I began to work on the back-end and code implementation of this application, it provided more clarity on what features I am actually able to support and how I may do this, and also things that I may have missed or misunderstood. Since I did not have this benefit early on, I had to rely entirely on the UML diagrams, sequence diagrams, and class diagrams to give me an idea of how the front-ends need to be structured.
- 2) **Dependencies:** I needed to conceptualize how I will be storing and interacting with data before I could begin designing my app, as this was the foundation on which I was building it on. I decided to stick with Guice because I was already familiar with it and did not want to create more challenges when the workload already seemed enormous, but the database gave me a lot of trouble early on. I tried to work with SQLite and various versions of it, but I always encountered versioning conflicts and spent a lot of time rebuilding and resyncing my application. Eventually I grew frustrated and pivoted to using something new: H2. I had researched embedded databases because I did not want to use something that relied on providing extra scripts or overcomplicating the deployment of the application. Since I saw that the H2 database could be entirely handled by my application and was relatively simple to implement, I decided upon using it and began to design my models in such a way as to accommodate it.
- 3) **Scope:** I could immediately see when I began constructing the frameworks for all my classes that there would be dozens of class files and this was very overwhelming. I did not know where to start or how to organize and keep track of everything. Most of my initial implementation had to be refactored because I wasn't getting it to work correctly or following best practices. I tried to define what each class should do bit by bit, starting ground-up with the models, then the repositories, then the controllers. As I started this, a clearer picture began to form because I had some constraints to work with (my repositories and models in addition to their defined relationships in the UML). However, what really helped me overcome this technical challenge was to focus on one specific feature or core component at a time and focus exclusively on that. This method of decomposition of larger issues into smaller components not only helped me ignore the overwhelming scope of the problem and make incremental progress toward implementing a solution but also aligned with the modular scope of the application and ensuring that everything worked independently as much as possible.
- 4) **Testing:** Inevitably I encountered bugs during the testing of this application, and some runtime errors proved very tricky to pin down because of the interconnected nature of the application with

so many modular parts that played a role in actions being performed. It became a nightmare to debug some issues like the creation of reservations, because these required other services and repositories affecting other aspects of my application (like a reservation requiring a guest – is this a new guest or do we already have them stored in our db? How many rooms are they occupying – we need to create ReservationRooms for these, are the rooms available on the given date(s) when the user is making the reservation? How do we know this?). A common practice was to have to reboot the app from the beginning, repopulating its preloaded data (the rooms, hotel, services, etc) from the AppConfig method which would always run only if I had dropped all tables and it detects that the database is empty. Sometimes I was even forced to do this, because I realized that my entities did not define the relationships correctly and I had to reload everything to make adjustments. Using logs and the IntelliJ terminal was the most effective means to overcome this aspect as I could see stack traces and pinpoint exact lines in my code where things were behaving erratically.

- 5) **Integration:** Finally, when the individual components were designed and functional, I had to make sure that the app would run as intended from beginning to end. This involved further testing based on created user stories or scenarios to ensure that the app could accomplish what it set out to do.

Personal reflections

Sometimes it felt like designing this application was a full-time job, which was very difficult to balance with the workload from other courses which also tended to escalate toward the end of the semester. I think breaking the project down into more smaller parts, similar to how the front-end design implementation was required as the first step of the project a month before it was due, would have benefitted in spreading out the workload. This is also amplified by the dependence on certain aspects of Java programming that I did not have knowledge of until the later parts of the course (such as working with the databases, writing to files, security, etc). I think that I would have done a better job if this was something we had started near the beginning of the semester and continuously worked on throughout the course, and as we learned new aspects of Java programming, we could apply them by continuously building up our project. This would have also provided the possibility of receiving feedback on every step of the journey to make sure that we are on track with our implementations and can make any corrections immediately before progressing too far ahead.

Suggestions for improvement

Right now, this application works under the assumption that there is a single hotel entity named “Come On Inn” located in Toronto. If the hotel was to expand to a chain of many locations the application has the scalability to support this, but would probably need to rethink its implementation in some areas during the reservation process. The application also relies on the front-desk or concierge to facilitate a lot of the processes such as receiving payments for a reservation and checking out, while its great that the hotel reservation automates certain processes and centralizes all the data, this could be further improved

upon by full automation with automatic confirmation, payment reception, and so on. For an end to end flow, the app would also benefit from using something like power automate or web-based functions to send emails to guests for confirmations of reservation bookings, reminders of feedback submission, and so on. This would also be expanded to run at specific timed intervals to automatically generate and export reports such as the audit log history. As I did not have the time to do this, a real working app would also support multithreading and allow multiple admins/managers to be logged in simultaneously while accurately tracking their actions. It would also probably store Guest account data as well so that recurring guests can track their activity and loyalty point histories (currently only supported for administrators) and maybe apply for specific retention discounts. The room assignments are also automated, so a future implementation that would improve the hotel could be to allow the guests to pick out the specific rooms that they want (provided they're available).